AMAR ELSHAER DEV

Animated dev videos, made with code.

The step-by-step guide to the kit behind my videos: a rigged cartoon of you, lip-sync, word-by-word captions and a free local clone of your own voice. Windows & Mac.

INSIDE

01 What you'll build
02 Install on Windows & Mac
03 Your voice (and how to make it better)
04 Your character
05 How the lip-sync works
06 Make an episode
07 Let Claude Code do it
08 Publish & troubleshoot



Website	amarhassan.com
Instagram	@amar_elshaer_dev
TikTok	@amar_elshaer_dev
LinkedIn	Amar Hassan Elshaer
Code	github.com/FireAmun/remotion-dev-video-kit

CHAPTER 01

01 What you'll build

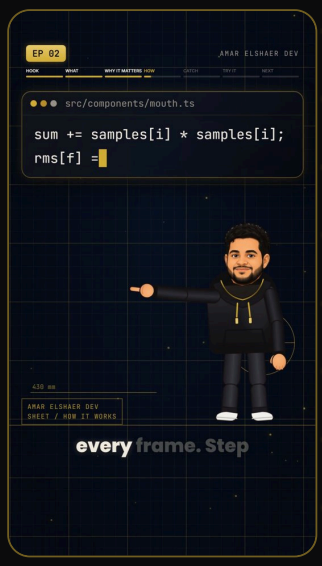
You write a short script. The kit turns it into a 30-60 second vertical video where a cartoon of you acts out every sentence, in your own (cloned) voice, with captions and sound effects. No camera, no editing app.



Split-flap hook



Live lip-sync



Blueprint code scene



Art-deco end card

The pipeline

Step	Tool	Output
1. Script	You (or Claude Code)	`narration.txt` , one paragraph per scene
2. Voice	Qwen3-TTS 1.7B, local, Apache 2.0	`narration.wav` at -16 LUFS
3. Word timings	faster-whisper (CPU)	`narration.json`
4. Scene plan	`plan.json` → `build_episode.py`	`episode.json` (validated)
5. Video	Remotion (React)	1080×1920, 30 fps MP4

> **Cost:** zero. Everything runs on your machine with free, open models. My setup: a Windows laptop with an RTX 3060 (6 GB). Rendering a 55 s video takes a few minutes.

CHAPTER 02

02 Install on Windows & Mac

You need **Node.js 18+** (for Remotion), **Python 3.11** (for the voice) and **Git**. A GPU makes the voice fast; without one it still works, just slower.

Windows

```
# tools (PowerShell)
winget install OpenJS.NodeJS.LTS
winget install Python.Python.3.11
winget install Git.Git

# the kit
git clone
https://github.com/FireAmun/remotion-
dev-video-kit
cd remotion-dev-video-kit
npm install

# Python env (NVIDIA GPU)
py -3.11 -m venv .venv-qwen
.venv-qwen\Scripts\python -m pip
install torch==2.6.0 torchaudio==2.6.0
--index-url
https://download.pytorch.org/whl/cu124
.venv-qwen\Scripts\python -m pip
install -r requirements.txt
```

No NVIDIA GPU? Install torch without `--index-url`; it runs on CPU.

Mac

```
# tools (Terminal, needs Homebrew:
brew.sh)
brew install node python@3.11 git

# the kit
git clone
https://github.com/FireAmun/remotion-
dev-video-kit
cd remotion-dev-video-kit
npm install

# Python env
python3.11 -m venv .venv-qwen
source .venv-qwen/bin/activate
pip install torch==2.6.0
torchaudio==2.6.0
pip install -r requirements.txt
```

Apple Silicon (M1-M4) uses the Mac GPU automatically. If the voice step errors on the GPU, run it with `PYTORCH_ENABLE_MPS_FALLBACK=1`. Intel Macs run on CPU (slow). I built and tested this on Windows; the Mac path uses the same code.

Check it works

```
npx remotion studio        # opens the preview in your browser
```

Open the `RigTest` composition (all 15 poses) and `StyleTest` (the style kit). On Mac, use `python` where this guide writes `.venv-qwen\Scripts\python`, after `source .venv-qwen/bin/activate`.

> **First voice run** downloads the Qwen3-TTS 1.7B model (about 4.2 GB) and faster-whisper small.en (about 0.45 GB). Delete models you stop using; they live in `~/.cache/huggingface`.

CHAPTER 03

03 Your voice

The voice is cloned "zero-shot": no training, just one short clip of you. **That clip decides everything**: the timbre, and also the energy and pace. A calm clip gives you a sleepy narrator.

Record the reference

1. Record **1-2 minutes** of yourself talking the way you want the videos to sound: like the start of a reel, energetic, smiling while you speak.
2. Quiet room, no echo (a room with clothes or curtains is great), no music. Phone or mic about a hand-width from your mouth.
3. Cut the **best 8-15 seconds**: clear, fluent, no "umm", no long pauses. Save it as `assets/voice/reference_energy.wav`.
4. Write exactly what you say in it, word for word (repeated words included), into `assets/voice/reference_energy.txt`.

Test it

```
.venv-qwen\Scripts\python scripts\tts_qwen.py public\episodes\test\narration.txt
```

How to make it sound more like you

Do this	Why
Try several clips from your recording and compare by ear	When I switched to a cleaner clip, my voice-match score went from 0.76 to 0.94 with the same model.
Pick the **most expressive** stretch (most pitch movement)	The clone copies your delivery. My first pick sounded "asleep"; the most expressive 11 s fixed it.
Use `!` on hook lines, commas for pauses	The model reads punctuation as energy and rhythm.
Write words the way they're **spoken**	`amar hassan dot com`, `N P X create video`, numbers as words. Captions still show the written form.
One paragraph per scene, under ~250 characters	Keeps every generation short and stable.
Keep the seed; change it only to re-roll a bad take	Same seed = repeatable takes.
Leave the levelling on (automatic)	The raw voice is quiet (~-36 dB). It's levelled to -16 LUFS so it's always louder than the sound effects.

> **Still not you?** Record more (20-30 min of clean narration) and fine-tune Qwen3-TTS on it (official scripts in the Qwen3-TTS repo; on a 6 GB GPU use LoRA, or a free cloud GPU), or simply record your own narration. **Only ever clone your own voice**, and keep the clip private (it's git-ignored).

CHAPTER 04

04 Your character

The character is a **puppet**: your cartoon head (two images: mouth closed and open) on a vector body with joints. That's what lets it point, type, celebrate and walk instead of being a static picture.

1. Make the art (Canva, ChatGPT or any image tool)

```
Prompt 1 (with a photo of you attached):
Flat vector cartoon of the man in the photo, same face, [hair], [beard], [glasses],
wearing a black hoodie with gold details. Front-facing, full body, standing,
friendly smile, mouth CLOSED. Plain solid light grey background, clean outlines.

Prompt 2 (attach the result of prompt 1):
Edit this exact image: keep everything identical and change ONLY the mouth
so it is open as if mid-speech, showing upper teeth.
```

2. Cut the head

```
.venv-qwen\Scripts\python scripts\make_head.py closed.png open.png ^
    --head 385 37 637 353 --mouth 470 255 590 300 ^
    --jaw 385,223 430,284 458,317 538,347 588,331 610,301 637,223
```

Read the pixel boxes off your image (Windows Paint or Mac Preview show the cursor position). The script removes the background, blends *only the mouth* from the open image onto the closed one (image tools redraw everything, so this stops the face from jittering), cuts at the jaw and cleans the edges.

3. Fit it to the body

- In `src/rig/Puppet.tsx` set `HEAD`: image size, `pivot` (bottom-centre of the chin) and the two eye boxes (used to draw real blinks).
- In `src/theme.ts` → `character`, set your skin tones and outfit colours (sample them from your art).
- Open `RigTest` in Studio and check every pose. Poses are just joint angles in `src/rig/poses.ts`.



RigTest: every pose of the rig

CHAPTER 05

05 How the lip-sync works

No AI. About 40 lines in `src/components/mouth.ts`. For every video frame it decides one thing: **mouth open or closed**.

1. **Measure** the loudness (RMS) of the audio in each frame: 30 checks a second, from the same audio you hear, so it can't drift out of sync.
2. **Compare** it to your loud speech (the top 10% of voiced frames), not to silence, so quiet and loud takes behave the same.
3. **Two thresholds**: open above 0.55, close below 0.40. The gap (hysteresis) stops the mouth from flickering.
4. Every mouth state lasts at least **2 frames**.

```
// per-frame loudness
for (let f = 0; f < frames; f++) {
  let sum = 0;
  for (let i = start; i < end; i++) sum += samples[i] * samples[i];
  rms[f] = Math.sqrt(sum / Math.max(1, end - start));
}
// reference = loud speech, not silence
const voiced = rms.filter((r) => r > peak * 0.02).sort((a, b) => a - b);
const loud = voiced[Math.floor(voiced.length * 0.9)] || 1;
// smooth, then open/close with hysteresis and a minimum hold
env += (level - env) * (level > env ? ATTACK : RELEASE);
const next = isOpen ? env ≥ MOUTH_CLOSE : env > MOUTH_OPEN;
if (next ≠= isOpen && held ≥ MIN_HOLD) { isOpen = next; held = 0; }
```

The audio comes from Remotion's `useAudioData()` (`@remotion/media-utils`). Because it's a pure function of the audio, every render is identical.

> **The catch:** it hears loudness, not sounds. Tools like Rhubarb Lip Sync recognise speech and map it to nine mouth shapes (6 basic + 3 extended). For a cartoon narrator, open/closed on loudness looks natural and never goes out of sync.

CHAPTER 06

06 Make an episode

Create a folder like `public/episodes/2026-01-01/` and work through these steps. Run one GPU job at a time (voice, then render).

1. **Script** → `narration.txt` : 30-60 s (≈ 75-150 words), one paragraph per scene, written as spoken. Hook in the first 2 seconds.
2. **Voice**: `.venv-qwen\Scripts\python scripts\tts_qwen.py public\episodes\<ep>\narration.txt`
3. **Word timings**: `.venv-qwen\Scripts\python scripts\timestamps.py public\episodes\<ep>\narration.wav --hint "Remotion, React"` (a few tech terms only; never the whole script, or Whisper starts repeating itself).
4. **Scenes** → `plan.json` : one scene per sentence, with a set, what the character does, and items that land on spoken words (`atWord`). Copy `public/episodes/example/plan.json` .
5. **Build**: `.venv-qwen\Scripts\python scripts\build_episode.py public\episodes\<ep>` then `npm run lint` .
6. **Preview** in `npx remotion studio` , fix what looks off, then **render**: `npx remotion render Episode out/<ep>.mp4 --codec=h264 --crf=18` (point `episodeFile` in `src/Root.tsx` at your folder).

A scene in plan.json

```
{
  "chapter": "HOOK",
  "display": "My mouth changed shape 196 times in yesterday's video!",
  "set": "stage",
  "actor": { "scale": 0.8, "beats": [{ "atMs": 0, "pose": "talk" },
                                     { "atWord": "196", "pose": "pointUp" }] },
  "items": [{ "kind": "splitflap", "text": "196", "label": "mouth moves",
              "atWord": "196", "x": 540, "y": 430 }]
}
```

Building blocks	
Sets	stage, coderoom, desk, sky (sunrise / timelapse), chalkboard, library, blueprint, plain
Poses	idle, talk, explain, shrug, wave, point, pointUp, pointViewer, celebrate, think, search, laptop, holdUp, thumbsUp, headScratch
Items	kinetic, splitflap, label, code, window (terminal / chalk), counter, stamp, steps, waveform, calendar, banner, confetti, book, icon, follow, end

CHAPTER 07

07 Let Claude Code do it

The repo ships a **Claude Code skill** (`.claude/skills/daily-video/`) that runs the whole chapter-06 workflow from a one-sentence idea, and a **CLAUDE.md** template with the video rules.

1. Fill in `.claude/CLAUDE.md` : your name, brand, handle, website, topics, posting time.
2. Open the project in Claude Code and say: *"make a video about <your idea>"*.
3. It researches (2-4 sources), then **stops** with a proposal: scene table, narration, facts with sources, captions. You approve or edit.
4. It makes the voice, builds the scenes, checks a frame of every scene itself, then **stops again** with a Studio preview.
5. After your OK it renders, writes the captions, and (with the Buffer connector) creates drafts for Instagram, TikTok and LinkedIn.

Rules the skill follows

- Same look every episode: only existing sets, poses and items in a daily run.
- ≤ 60 s, hook in 2 s, one sentence = one scene, a visual change every 1.5-3 s.
- Never invents facts or code; sources are stored with each episode.
- The voice is always the loudest thing; effects sit ~12 dB under it.
- Never posts anything without your approval.

CHAPTER 08

08 Publish & troubleshoot

Publishing

- **Buffer** (free plan): its API only accepts videos from a public link, so the skill creates drafts with captions and time already set; you attach the MP4 and press Schedule.
- Turn on the **AI-generated** label on Instagram and TikTok when the voice is a clone. It's honest and protects the account.
- Write a separate, more professional caption for LinkedIn.
- "Comment **DEV** and I'll send you the code" works great: reply in DMs with your repo link.

Troubleshooting

Problem	Fix
Whisper transcript repeats the same sentence	Don't pass the script as a prompt; use `--hint` with a few terms.
Captions start early after a pause	Handled: scenes that start a narration paragraph take the exact time from the voice step.
Video longer than 60 s	Shorten the narration, not the speed. Check the length right after the voice step.
Sound effects louder than the voice	Make sure the voice went through `scripts/audio_level.py` (automatic in `tts_qwen.py`).
A word is mispronounced	Spell it as spoken in `narration.txt` (`T-T-S`, `dot com`); keep the written form in `plan.json`.
Item covers the character's face	Move the item, shrink the actor (`scale`), or put him at the side (`x: 790, facing: "left"`).
Studio stutters	Preview only; the final render is frame-perfect.

Credits & licenses

Kit code and docs: MIT. Remotion has its own license (free for individuals and small teams; companies may need one: remotion.dev/license). Qwen3-TTS: Apache 2.0. faster-whisper: MIT. Sound effects: CC0 (Kenney via soundcn). The example character is my likeness: replace it with yours before you publish.

Built something with it? Tag me.

amarhassan.com · @amar_elshaer_dev on Instagram & TikTok · Amar Hassan Elshaer on LinkedIn